

HaskellMOOC.fi Notes

Jiang and Sharma

<2026-05-19 Tue>

Contents

I	Lecture 1: And so It Begins	2
1	Haskell Characteristics	3
2	Fundamental Concepts	4
3	Program Structure	5
II	Lecture 2: Recursion & Lists	6
4	Recursion & Control Flow	7
5	Lists & Polymorphism	8
6	Handling Optionality	9
III	Lecture 3: Higher-Order Functions	10
7	Functions as First-Class Citizens	11
8	Composition & Lambdas	12
9	Advanced List Patterns	13
IV	Lecture 4: Type Classes & Folds	14
10	Tuples & Folds	15
11	Type Classes	16

V	Lecture 5: Algebraic Datatypes (ADTs)	17
12	Defining Custom Types	18
13	Recursive Types & Trees	19
14	Record Syntax	20
VI	Lecture 6: Working Class Hero	21
15	Lecture Contents:	22

Part I

Lecture 1: And so It Begins

Chapter 1

Haskell Characteristics

- Functional, pure, and lazy
- Strongly typed with type inference
- Garbage collected and compiled
- **Tooling:** ‘stack ghci’ (REPL), ‘:t’ (type), ‘:load’, ‘:reload’

Chapter 2

Fundamental Concepts

- **Expressions:** Everything is an expression (no statements)
- **Application:** `'f x y'` (not `'f(x,y)'`); function calls bind tightest
- **Basic Types:** `'Int'`, `'Integer'`, `'Double'`, `'Bool'`, `'String'` (`'[Char]'`)
- **Currying:** `'a -> b -> c'` represents multi-argument functions

Chapter 3

Program Structure

- **Modules:** One per file
- **Definitions:** Optional type signature + equations
- **Logic:** ‘if...then...else’ (else is mandatory), ‘where’/‘let’
- **Immutability:** Bindings are constant; “updates” create new values

Part II

Lecture 2: Recursion & Lists

Chapter 4

Recursion & Control Flow

- **Helper Functions:** Often marked with ‘‘ (e.g., ‘fibonacci’’) to manage state
- **Tail Recursion:** Recursive call is the last action → optimizes to a loop
- **Guards:** ‘| condition = value‘ (first true guard wins)

Chapter 5

Lists & Polymorphism

- **Lists:** Homogeneous `[a]`, singly-linked, constructed via `cons` `·:`
- **Common Ops:** `head`, `tail`, `take`, `drop`, `++`, `!i`, `reverse`
- **Polymorphism:** Lowercase type variables (e.g., `a`) determined by unification

Chapter 6

Handling Optionality

- **Maybe a:** ‘Nothing | Just x’ (Safe alternative to null)
- **Either a b:** ‘Left a | Right b’ (Commonly used for success/error)
- **Case expressions:** ‘case ... of’ for complex pattern matching

Part III

Lecture 3: Higher-Order Functions

Chapter 7

Functions as First-Class Citizens

- **HOFs:** Functions that take or return functions (e.g., ‘map’, ‘filter’)
- **Partial Application:** ‘add 3’ returns a function waiting for the second arg
- **Sections:** Partially applied operators, e.g., ‘(+1)’, ‘(*2)’

Chapter 8

Composition & Lambdas

- **Composition:** $(f . g) x = f (g x)$
- **Application Operator:** '\$' (low precedence, avoids parentheses)
- **Lambdas:** Anonymous functions $\lambda x. x + 1$

Chapter 9

Advanced List Patterns

- **Comprehensions:** `[x | x <- xs, cond]`
- **Recursion Patterns:** Building via `⋮` and consuming via `x:xs`
- **Typed Holes:** Using `_` to let GHC suggest missing types

Part IV

Lecture 4: Type Classes & Folds

Chapter 10

Tuples & Folds

- **Tuples:** Fixed-length, mixed types; accessed via ‘fst’/‘snd’
- **foldr:** “Crunches” a list into a single value
 - ‘foldr f z [] = z’
 - ‘foldr f z (x:xs) = f x (foldr f z xs)’

Chapter 11

Type Classes

- **Definition:** Groups of types sharing common operations (not OOP classes)
- **Constraints:** `'(Eq a) => ...'` restricts polymorphism to types that implement `'Eq'`
- **Key Classes:**
 - `'Eq'` ($\approx$, $\neq$) and `'Ord'` ($<$, $>$, `'compare'`)
 - `'Num'` $\rightarrow$ `'Integral'` / `'Fractional'` $\rightarrow$ `'Floating'`
 - `'Show'` (to string) and `'Read'` (from string)

Part V

Lecture 5: Algebraic Datatypes (ADTs)

Chapter 12

Defining Custom Types

- **ADTs:** Sum of constructors, product of fields (`'data T = C1 | C2 Int String'`)
- **Deriving:** Automatic implementation of `'Show'`, `'Eq'`, `'Read'`
- **Type Parameters:** Generic types, e.g., `'data Maybe a'`

Chapter 13

Recursive Types & Trees

- **Recursive Types:** Types that refer to themselves (e.g., custom ‘List a’)
- **Trees:** ‘Node val left right | Empty’
- **Operations:** Height calculation, BST lookup, and insertion

Chapter 14

Record Syntax

- Named fields instead of positional arguments
- Automatically generates accessor functions
- Improves readability and maintainability for large data structures

Part VI

Lecture 6: Working Class Hero

Chapter 15

Lecture Contents:

- Defining new Type Classes
- Instantiating class using *instance* and *deriving* keywords